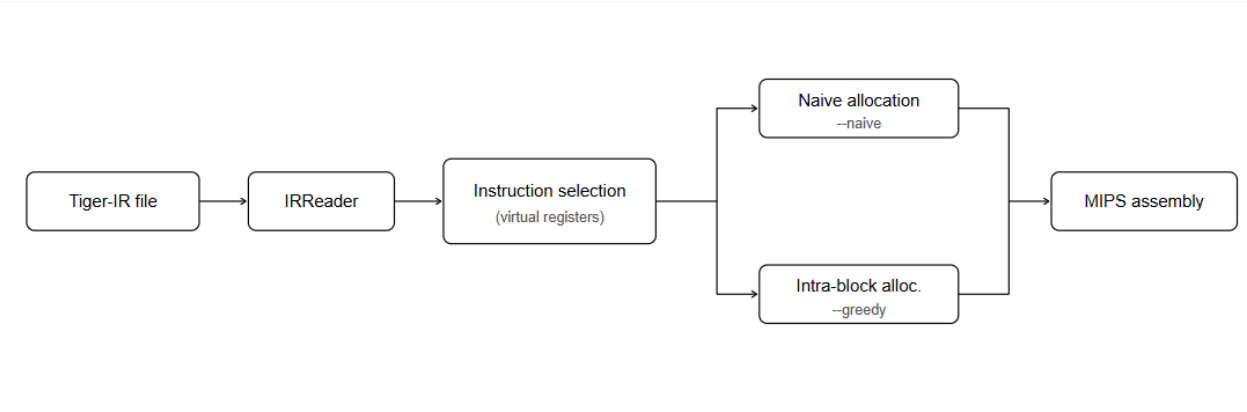
Code Implementation Structure & Decisions



Instruction Selection

High Level Architecture
Each TigerIR instruction is translated into a MIPS instruction using virtual registers. There are unlimited virtual registers, and they are named $varname. The allocator maps them to physical registers later. When needed, constants are stored in registers called $tmp0, $tmp1,....etc. Array address computation uses separate virtual temporaries ($__addr0,....) to hold the computed address before a load or store instruction.

Having the register names be $varname, $tmp0, $__addr0 makes it easy to implement temporary registers, and it is not complicated to assign values to registers. In addition, having this modular design where instruction selection and allocation are handled separately also reduces the complexity in our code, and means that instruction selection does not have to handle physical registers.

Low Level Design Decisions

1) Sub with constant uses ADDI instruction with a negated value. This avoids needing an LI and SUB instruction.
2) Arguments are pushed right-to-left so that they appear in the correct order on the stack.
3) There is a helper method called resolveToRegister that loads imm values into a fresh virtual temporary $tmpN. This means that for every instruction that can't use an imm value, there is a simple and consistent way to handle it.
4) In IR, labels are treated like their own instruction; however, in MIPS, they are not there not their own instruction, so labels are stored until the next valid instruction is emitted, and they are printed with the instruction.

```java
/**
 * Translates {@code instr} and emits the resulting MIPS instruction(s).
 *
 * @param instr the IR instruction to lower
 */
public void generateInstructions(IRInstruction instr) {
  switch (instr.opCode) {
    case ASSIGN -> emitAssign(instr);
    case ADD -> emitAdd(instr);
    case SUB -> emitSub(instr);
    case MULT -> emitBinaryRR(instr, MIPSOp.MUL);
    case DIV -> emitBinaryRR(instr, MIPSOp.DIV);
    case AND -> emitBinaryRR(instr, MIPSOp.AND);
    case OR -> emitBinaryRR(instr, MIPSOp.OR);
    case GOTO -> emit(new MIPSInstruction(MIPSOp.J, label: null,
        new Addr(instr.operands[0].toString())));
    case BREQ -> emitBranch(instr, MIPSOp.BEQ);
    case BRNEQ -> emitBranch(instr, MIPSOp.BNE);
    case BRLT -> emitBranch(instr, MIPSOp.BLT);
    case BRGT -> emitBranch(instr, MIPSOp.BGT);
    case BRGEQ -> emitBranch(instr, MIPSOp.BGE);
    case RETURN -> emitReturn(instr);
    case CALL -> emitCall(instr);
    case CALLR -> emitCallR(instr);
    case ARRAY_STORE -> emitArrayStore(instr);
    case ARRAY_LOAD -> emitArrayLoad(instr);
    case LABEL -> {
      String labelName = instr.operands[0].toString();
      if (pendingLabel != null) {
        emit(new MIPSInstruction(MIPSOp.J, label: null, new Addr(labelName)));
      }
      labels.put(labelName, mipsIdx);
      pendingLabel = labelName;
    }
  }
}
```

Figure 1: The main entry to instruction selection, taking an IR Instruction and producing the appropriate MIPS Assembly with register resolution depending on the `opCode`

Note: Each instance of InstructionSelection is specific to the singular IR instruction passed into it (not a singular instance for the entire function)

```java
/**
 * Returns a virtual register holding {@code op}'s value. If {@code op} is a
 * constant, a fresh temporary is materialized with {@code li}; otherwise the
 * operand's existing virtual register is returned directly.
 */
private Register resolveToRegister(IROperand op) {
  if (op instanceof IRConstantOperand c) {
    String tmp = "$tmp" + tmpCounter++;
    if (c.type instanceof IRFloatType) {
      emit(new MIPSInstruction(MIPSOp.LI_S, label: null,
          new Register(tmp, isVirtual: true),
          new Imm(op.toString(), type: "SINGLE")));
    } else {
      emit(new MIPSInstruction(MIPSOp.LI, label: null,
          new Register(tmp, isVirtual: true),
          new Imm(op.toString(), type: "DEC")));
    }
    return new Register(tmp, isVirtual: true);
  }
  return new Register("$" + op.toString(), isVirtual: true);
}
```

Figure 2: The `resolvetoRegister` function utilized for each MIPS assembly instruction being emitted resolves variables to their appropriate virtual registers.

Naive Allocation

The `NaiveAllocator` class is the register allocation pass that comes after `InstructionSelection`. It follows a simple strategy of unconditionally spilling values onto the stack. Physical registers are only used as temporary scratch registers to load, compute, then store around each instruction. This ensures correctness, at the expense of performance due to the continuous round-tripping through memory.

Stack Frame Layout

Our stack frames follow a conventional layout during function calls, with arguments saved first with offsets of 4 bytes from arg n down to arg 0. Following those at lower addresses we have the saved return address (`$ra`) and the saved frame pointer (`$fp`). Below those, we have the virtual register slots that are stored on the stack following the same byte offset convention as arguments (4 byte offsets each).

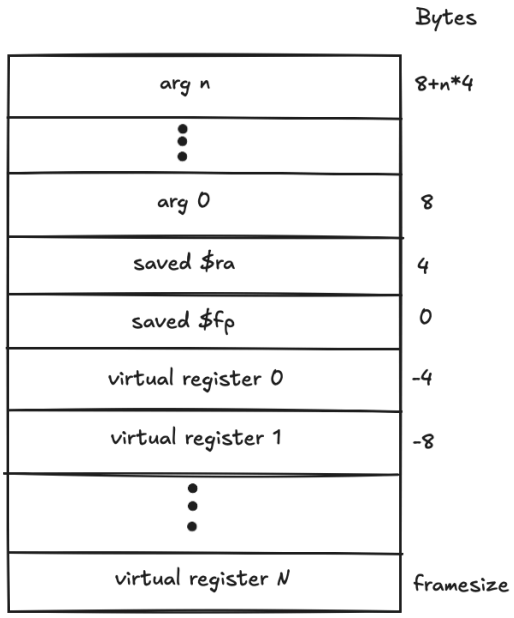


Figure 3: Example stack frame layout for our `NaiveAllocator` class

allocate()

```java
public void allocate(IRFunction func, Map<Integer, MIPSInstruction> inputInstructions, String trailingLabel) {
  Register sp = new Register(name: "$sp", isVirtual: false);
  Register fp = new Register(name: "$fp", isVirtual: false);
  Register ra = new Register(name: "$ra", isVirtual: false);

  emitPrologue(sp, fp, ra);

  Map<String, Integer> vrMap = buildVrMap(inputInstructions);
  int frameSize = 4 * vrMap.size();

  Set<String> paramNames = new HashSet<>();
  for (IRVariableOperand p : func.parameters)
    paramNames.add(p.getName());

  int totalArrayDataSize = 0;
  List<IRVariableOperand> localArrayVars = new ArrayList<>();
  for (IRVariableOperand var : func.variables) {
    if (var.type instanceof IRArrayType && !paramNames.contains(var.getName())) {
      localArrayVars.add(var);
      totalArrayDataSize += ((IRArrayType) var.type).getSize() * 4;
    }
  }

  Map<String, Integer> arrayBaseOffsets = new LinkedHashMap<>();
  int nextArrayBase = -(frameSize + totalArrayDataSize);
  for (IRVariableOperand var : localArrayVars) {
    String key = "$" + var.getName();
    arrayBaseOffsets.put(key, nextArrayBase);
    nextArrayBase += ((IRArrayType) var.type).getSize() * 4;
  }

  emit(new MIPSInstruction(MIPSOp.ADDI, label: null, sp, sp,
      new Imm(String.valueOf(-(frameSize + totalArrayDataSize)), type: "DEC")));

  loadParameters(func, vrMap, fp);
  initArrays(arrayBaseOffsets, vrMap, fp);
  Set<String> floatVrs = buildFloatSet(inputInstructions);
  String epilogueLabel = func.name + "_epilogue";
  emitBody(inputInstructions, vrMap, fp, epilogueLabel, trailingLabel, floatVrs);
  emitEpilogue(sp, fp, ra, epilogueLabel);
}
```

Figure 4. allocate function from our `NaiveAllocator` class

The `allocate()` function's logic can be understood in phases:

1. Create the `$sp`, `$fp`, and `$ra` registers and then call `emitPrologue` which saves the `$ra` and `$fp` onto the stack and updates `$fp` = `$sp`
2. Next a call is made to `buildVrMap` which takes our map containing the generated MIPS instructions, identifies all of the contained virtual registers, and spills them onto the stack at the next available offset from the `$fp`, decrementing the offset by 4 bytes each relative to the `$fp`; the result is a map that maps virtual register names -> their relative offset from the `$fp` in the stack frame (Note: This only builds a virtual mapping, and does NOT place any values onto the stack yet)
3. Using this map, we can calculate the framesize needed to update our `$sp`, decrementing by framesize + totalArrayDataSize to make room for all of our coming spill values. Two initialization steps then occur:
    a. `loadParameters`: Loads arguments stored above the `$fp` into their appropriate spill slots
    b. `initArrays`: For any declared arrays, stores the address of where the array's data lives (on the stack) within its appropriate spill spot
4. `emitBody` then takes each virtual instruction, loads its source registers from the stack into physical scratch registers, then emits those instructions using those physical registers, storing the result back onto the stack

5. `emitPrologue` cleans up the stack frame by restoring the `$sp` from `$fp` in order to collapse the stack frame, then pops `$fp` and `$ra` off of the stack and finally executes jump return to return to `$ra` (the caller)

Why this approach?

This approach ensures correctness while maintaining simplicity as we do not need to worry about register allocation (liveness analysis, graph coloring, greedy, etc.) as all arguments are spilled (stored) on the call stack, at specific known offsets. Making loading, computation, and storage very simple to code and understand.

Intrablock Allocation (Greedy Allocation)

The `GreedyAllocator` class is the second method of register allocation, which is invoked with the `--greedy` flag. This register allocation algorithm improves on the naive register allocation in NaiveAllocator, by keeping the most-used virtual registers in physical registers **across an entire basic block**, rather than reloading them from the stack from every instruction (as is done through the naive register allocation). The most used virtual registers need only to be loaded once at the entry of the basic block and are stored once at the exit of the basic block, instead of going to memory on every instruction, costing more unnecessary loads. An important note is that the greedy allocator uses the same stack-frame layout as the naive allocator. Every virtual register is still assigned a fixed $fp-relative stack slot, and local array data is stored below the virtual-register spill area. The difference is in how the function body uses those slots. The naive allocator loads operands before almost every instruction and stores results immediately afterward. The greedy allocator instead keeps selected virtual registers in physical registers for the duration of a basic block, loading them at block entry when needed and storing them back at block exit. Virtual registers that are not selected are still handled with the naive per-instruction load/store pattern.

Register Considerations

We noted earlier on in our design process the following line: “You may also need to reserve some registers to temporarily hold the values for spilled registers when they are loaded from the stack”. Following the project specification's guidance, we split the ten allowed temporary registers ($t0–$t9) into two roles: **selected**, which hold the most-used virtual registers across an entire block, and **scratch**, which temporarily hold spilled values within a single instruction.

Selected Virtual Registers:

These virtual registers are the ones that we will keep in a physical register for the whole basic block. They will get loaded once at the entry of the basic block, used directly (i.e., without loading from the stack) in every instruction that touches them, and stored once at block exit. In

our implementation, we have reserved registers $t0 through $t7 to hold the virtuals with the highest use count. If there are ties, whichever appeared first in the block wins.

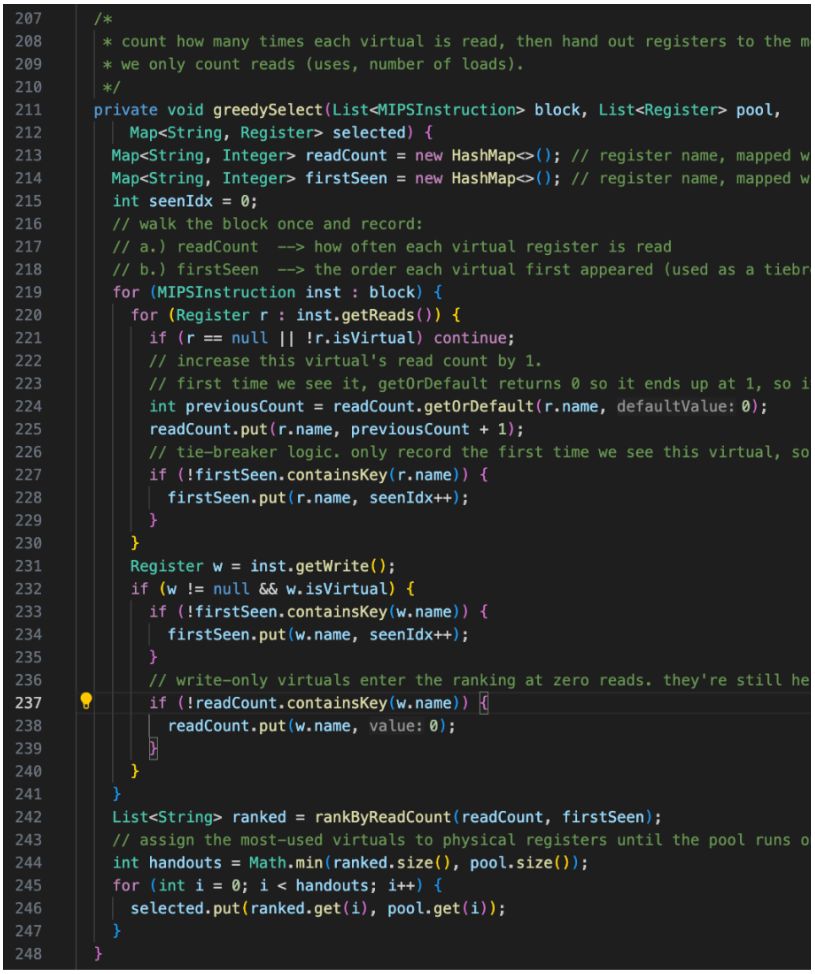
Scratch Virtual Registers

Drawing particular attention to that quote where we need to reserve some physical registers to temporarily hold the values for scratch registers when they are loaded from the stack, we have elected to choose registers $t8 through $t9 in order to temporarily hold these values. The idea is that these registers should be freshly available for each successive MIPS instruction. Since MIPS uses a three-address ISA with at most two source operands per instruction, two scratch registers are enough to cover any instruction whose sources are both spilled. We can elect to replace one of the scratch registers, which were used as a source for the instruction, as the destination register such as: add $t8, $t8, $t9.

Greedy Allocation Algorithm Walkthrough

For each of the basic blocks, the greedy allocator performs the following phases:

1. Count the number of virtual register uses: The allocator scans the basic block and counts how many times each virtual register is read. Virtual registers with more reads are more likely to be kept in physical registers because it then avoids unnecessary memory loads.
2. **Greedily** select the physical registers: The allocator sorts virtual registers by read count and assigns the most-used virtual registers to $t0 through $t7. Virtual registers that do not receive a physical register are treated as spilled and will be assigned to either $t8 or $t9 before their instruction executes.
3. Load selected values: At the start of the block, the allocator loads selected virtual registers only if the first use is a read. If a virtual register is written to before it is read, it does not need an entry load because the block will define its value later on.
4. Rewrite each instruction with relevant assignments: For each instruction, selected virtual registers are replaced directly with their assigned physical registers. Spilled virtual registers temporarily borrow scratch registers being the registers: $t8 and $t9. If an instruction has two spilled reads and one spilled write, the spilled write can reuse one of the read scratch registers as its destination, since MIPS allows definitions and uses to share a physical register like: add $t8, $t8, $t9.
5. Store spilled writes immediately: If an instruction writes to a spilled virtual register, the result is stored back to that virtual register's stack slot immediately after the instruction. This keeps memory up to date for spilled values.
6. At the end of a basic block, store the value currently in a physical register back into its stack slot in memory: The allocation is purely intrablock (within a basic block), therefore use counts can differ per block, hence our motivation behind this step. The next block may assign the same virtual register to a different physical register, or may not keep it in a register at all. Thus, before leaving the block, the allocator stores all selected registers that were used or modified back to their stack slots.



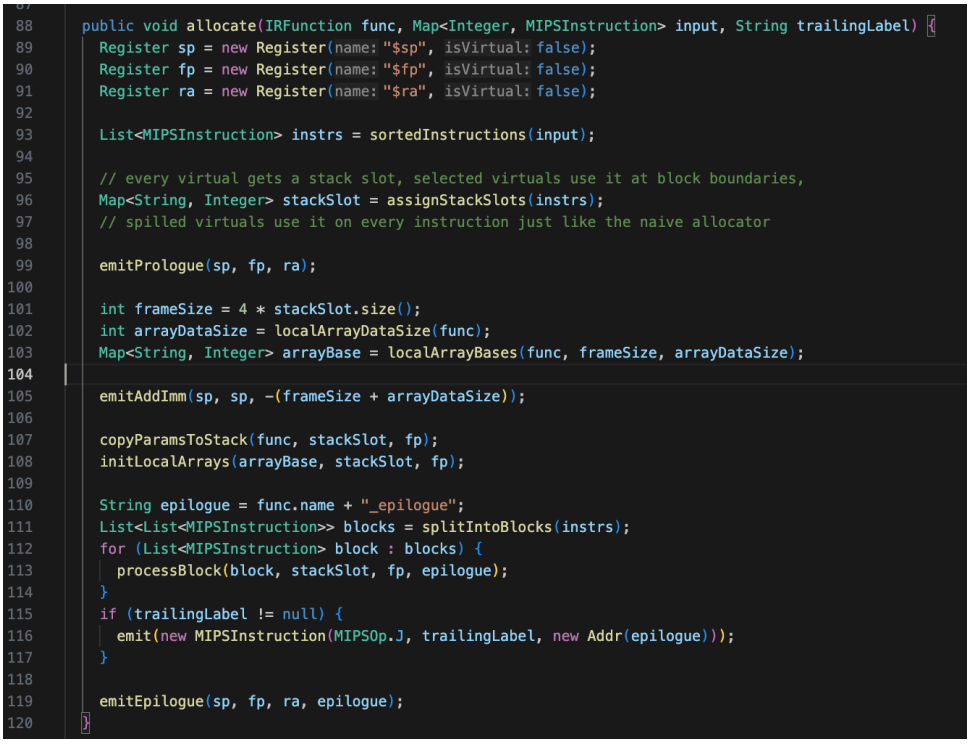
```
207  /*
208   * count how many times each virtual is read, then hand out registers to the m
209   * we only count reads (uses, number of loads).
210   */
211  private void greedySelect(List<MIPSInstruction> block, List<Register> pool,
212      Map<String, Register> selected) {
213    Map<String, Integer> readCount = new HashMap<>(); // register name, mapped w
214    Map<String, Integer> firstSeen = new HashMap<>(); // register name, mapped w
215    int seenIdx = 0;
216    // walk the block once and record:
217    // a.) readCount  --> how often each virtual register is read
218    // b.) firstSeen  --> the order each virtual first appeared (used as a tiebr
219    for (MIPSInstruction inst : block) {
220      for (Register r : inst.getReads()) {
221        if (r == null || !r.isVirtual) continue;
222        // increase this virtual's read count by 1.
223        // first time we see it, getOrDefault returns 0 so it ends up at 1, so i
224        int previousCount = readCount.getOrDefault(r.name, defaultValue: 0);
225        readCount.put(r.name, previousCount + 1);
226        // tie-breaker logic. only record the first time we see this virtual, so
227        if (!firstSeen.containsKey(r.name)) {
228          firstSeen.put(r.name, seenIdx++);
229        }
230      }
231      Register w = inst.getWrite();
232      if (w != null && w.isVirtual) {
233        if (!firstSeen.containsKey(w.name)) {
234          firstSeen.put(w.name, seenIdx++);
235        }
236        // write-only virtuals enter the ranking at zero reads. they're still he
237        if (!readCount.containsKey(w.name)) {
238          readCount.put(w.name, value: 0);
239        }
240      }
241    }
242    List<String> ranked = rankByReadCount(readCount, firstSeen);
243    // assign the most-used virtuals to physical registers until the pool runs o
244    int handouts = Math.min(ranked.size(), pool.size());
245    for (int i = 0; i < handouts; i++) {
246      selected.put(ranked.get(i), pool.get(i));
247    }
248  }
```

Figure 5: The `greedySelect` function, which assigns physical registers to virtual registers within a single basic block. It performs the read/use counting pass through the basic block and then hands out registers from the physical register pool to the most-read virtual registers (lines 242–247).

The `greedySelect()` function decides which virtual registers in a block win one of the physical registers in our physical register pool (`$t0`–`$t7`). Its logic can be understood in phases:

1. First, two empty maps and a counter are created to track information about each virtual register encountered in the block: `readCount` (which records how many times each virtual register is read in the block), `firstSeen` (which records the order each virtual first appears in the block, used later as a tiebreaker), and a `seenIdx` counter that increments each time we discover a new virtual (which helps identify ordering for tiebreaking).
2. Next, a single walk through every instruction in the block is performed. For each instruction, the read operands are inspected first:

    1. If the operand is a virtual register, its count in `readCount` is incremented by one (using `getOrDefault` so the first time we see a virtual it ends up at 1, ensuring that it is still counted).
    2. If we haven't seen this virtual before, an entry is made in `firstSeen` with the current value of `seenIdx`, so we have a deterministic tiebreaker for sorting later.
3. After the reads have been processed, the write operand of the instruction is inspected. If the write is a virtual register that hasn't been seen yet, it is registered in `firstSeen` and added to `readCount` with a value of 0. This handles the special case of write-only virtual registers: they enter the ranking at zero reads, so they still appear in the ranked list (and can participate in tiebreaking), but they always lose to any virtual that is actually read by the initial incrementing.
4. Once the entire block has been walked, a call is made to `rankByReadCount` which sorts the virtual registers by descending read count, breaking ties first by `firstSeen` order, and breaking any remaining ties alphabetically by name. The result is a list of virtual register names in priority order, with the virtual registers with the highest priority for a physical register first.
5. Finally, physical registers from the physical register pool are handed out to the top virtual registers in the ranked list. The number of handouts is capped at `min(ranked.size(), pool.size())`, so once the physical register pool runs out, every remaining virtual in the ranked list is left unselected and will be treated as spilled when the block is emitted.



```java
public void allocate(IRFunction func, Map<Integer, MIPSInstruction> input, String trailingLabel) {
  Register sp = new Register(name: "$sp", isVirtual: false);
  Register fp = new Register(name: "$fp", isVirtual: false);
  Register ra = new Register(name: "$ra", isVirtual: false);

  List<MIPSInstruction> instrs = sortedInstructions(input);

  // every virtual gets a stack slot, selected virtuals use it at block boundaries,
  Map<String, Integer> stackSlot = assignStackSlots(instrs);
  // spilled virtuals use it on every instruction just like the naive allocator

  emitPrologue(sp, fp, ra);

  int frameSize = 4 * stackSlot.size();
  int arrayDataSize = localArrayDataSize(func);
  Map<String, Integer> arrayBase = localArrayBases(func, frameSize, arrayDataSize);

  emitAddImm(sp, sp, -(frameSize + arrayDataSize));

  copyParamsToStack(func, stackSlot, fp);
  initLocalArrays(arrayBase, stackSlot, fp);

  String epilogue = func.name + "_epilogue";
  List<List<MIPSInstruction>> blocks = splitIntoBlocks(instrs);
  for (List<MIPSInstruction> block : blocks) {
    processBlock(block, stackSlot, fp, epilogue);
  }
  if (trailingLabel != null) {
    emit(new MIPSInstruction(MIPSOp.J, trailingLabel, new Addr(epilogue)));
  }

  emitEpilogue(sp, fp, ra, epilogue);
}
```

Figure 6: The `allocate()` method of `GreedyAllocator`, which is the entry point for the greedy register allocation pass. It builds the per-virtual stack slot map (`assignStackSlots`), emits the function prologue, partitions the instructions into basic blocks (`splitIntoBlocks`), and then delegates the per-block work to `processBlock` before emitting the shared epilogue.

The `allocate()` function's logic can be understood in phases:

1. Create the `$sp`, `$fp`, and `$ra` registers and then call `emitPrologue` which saves the `$ra` and `$fp` onto the stack and updates `$fp = $sp` (the same prologue setup as the naive allocator).
2. Next a call is made to `sortedInstructions` which takes the map of generated MIPS instructions and returns them as a list ordered by their original emission index. Working with an ordered list is necessary because basic-block boundaries depend on the original instruction sequence.
3. A call is then made to `assignStackSlots` which scans the ordered instruction list, identifies all of the contained virtual registers, and assigns each one a fixed offset on the stack relative to the `$fp`, decrementing the offset by 4 bytes each time a new virtual is encountered. The result is a map of virtual register names → their relative offset from the `$fp` in the stack frame (Note: This only builds a virtual mapping, and does NOT place any values onto the stack yet. Every virtual register gets a slot regardless of whether it later wins a physical register, since selected virtual registers also need a stack home for block-entry loads and block-exit flushes). The stack layouts for both allocators are identical.
4. Using this map, we can calculate the framesize needed to update our `$sp`, decrementing by `framesize + arrayDataSize` to make room for all of our coming spill values. Two initialization steps then occur:
    1. `copyParamsToStack`: Loads arguments stored above the `$fp` into their appropriate spill slots
    2. `initLocalArrays`: For any declared arrays, stores the address of where the array's data lives (on the stack) within its appropriate spill spot
5. Next a call is made to `splitIntoBlocks` which partitions the ordered instruction list into basic blocks. A block starts at any label (a possible branch target) and ends at any branch, jump, or call (calls end blocks because the callee can clobber any selected register).
6. For each basic block, `processBlock` is invoked, which performs the per-block planning (including the `greedySelect` call shown in Figure 5), emits any necessary entry loads, translates each instruction in the block using selected and scratch registers, and flushes dirty selected registers back to their stack slots before the block ends.
7. `emitEpilogue` cleans up the stack frame by restoring the `$sp` from `$fp` in order to collapse the stack frame, then pops `$fp` and `$ra` off of the stack and finally executes a `jr $ra` to return to the caller.

Low Level Design Decisions:

1. Every virtual register also gets a stack slot, even ones that win a selected register. This guarantees that block-exit flushes always have a target, and lets the allocator fall back to memory in any edge case. The slot map is built up front in `assignStackSlots` before blocks are assigned.
2. Selection is per-block, not per-function. A virtual that is selected into $t0 in one block can be spilled or assigned to a different physical register in the next, depending on its use count in each block. This gives every block the freedom to optimize for its own access pattern.
3. We only flush selected registers we actually touched. A virtual that won a selected slot but was never read or written in the block has the same value in the register and on the stack, so flushing it would be wasted work (or worse, write garbage if we never loaded the register in the first place). A dirty set tracks which selected virtual registers we loaded or wrote to, and `flushToStack` only writes back those.
4. We reuse scratch registers for spilled writes. With only two integer scratch registers (`$t8` and `$t9`), an instruction whose two source operands and destination are all spilled would naively require three distinct scratch registers. Instead of increasing the number of scratch registers, we let the destination reuse one of the scratch registers that already holds a spilled read, since MIPS allows a destination and source to share a register (e.g. `add $t8, $t8, $t9` is legal). Once both reads have been emitted, overwriting one of them with the result is safe because no source value remains to preserve. The reuse logic lives in `scratchForWrite`, which first checks whether a fresh scratch register is available and only falls back to reusing a borrowed one if both are in use. This trick is what frees `$t7` for another entry in the physical register pool, giving every block one extra register to keep more frequently used virtual registers ready.
5. `JR $ra` is rewritten as `J <epilogue>`. A naive translation of IR `RETURN` would become `JR $ra`, but by the time we reach a return, the function has already pushed `$ra` and `$fp` and reserved space for spills and local arrays. Jumping straight to `JR $ra` would skip all of that teardown and leave the caller on top of a corrupted frame. Instead, every function emits a single shared `<func_name>_epilogue` block that restores `$sp`, pops `$fp` and `$ra`, and only then performs the real `jr $ra`. Every `JR` inside the function body is rewritten as `J <epilogue>` so all returns route through this one cleanup path, which means the teardown code is written only once instead of duplicated at every return point.

Why This Approach?

This approach strikes a balance between the simplicity of the naive allocator and the complexity of full graph-coloring register allocation with K registers. The greedy strategy needs only an intra-block read count to make its decisions, with no global liveness analysis, no interference graph, and no coloring heuristic. Even though the strategy is block-local, the results prove that this is a very effective approach, because the most frequently accessed virtual registers in any given block are exactly the ones whose constant reloading hurts performance the most under

the naive scheme. By keeping a stack slot for every virtual register as a backup, we also preserve the naive allocator's correctness guarantees: spilled virtual registers behave identically to their naive counterparts, and selected virtual registers are simply loaded into physical registers and stored at block boundaries rather than around every instruction. The result is a register allocator that is straightforward to implement and debug, while still cutting load instructions roughly in half on the test cases (see Performance Evaluation below).

Software Challenges (yall feel free to add this)

1) We did not consider the assign array instruction and had implemented assign to only have 2 operands.
2) `addi` utilizes 16-bit immediate values to create space for arrays, but when working with arrays that were larger than signed 32,767, needed to use `load immediate` instead
3) For the greedy allocator, before the fix, virtual registers with the same read count could end up assigned to different physical registers depending on how Java's HashMap decided to iterate that run. The output was always correct but the assembly text changed across runs. We fixed `rankByReadCount` to break ties first by the order each virtual first appeared in the block, then alphabetically by name, so the compiler respects input-output equivalency across runs.

Known Bugs/Deficiencies

As of the time of writing this, there are no known implementation bugs, and our implementation meets the requirements for the number of instruction reads (loads) across public tests and internal team testing.

Run/Build Script

build.sh

- Java JDK 17 or higher is required to compile

run.sh

- args[0] - path to the input Tiger-IR file
- args[1] - path to output .s assembly file
- args[2] - --naive for naive register allocation
  --greedy for intra-block greedy register allocation

Performance Evaluation with Public Test Cases

Quicksort

We ran both naive and greedy allocation on quicksort and using spims determined that for inputs 0 - 9, there is an average of a 42.5% reduction in load instructions.

Input	Naive Reads	Greedy Reads	Reduction
0	1101	667	434 (39.4%)
1	2587	1518	1069 (41.3%)
2	4181	2427	1754 (42.0%)
3	5826	3347	2479 (42.6%)
4	7231	4162	3069 (42.4%)
5	9683	5464	4219 (43.6%)
6	10785	6141	4644 (43.1%)
7	12880	7291	5589 (43.4%)
8	14570	8236	6334 (43.5%)
9	16284	9195	7089 (43.5%)
Avg			42.5%

Figure 7. Naive vs Greedy Performance on Quicksort

Prime

This table shows the number of reads for naive and greedy allocators. It shows that there was an average of a 43.9% reduction in load instructions. Input 0 is not included as it only showed a 25% decrease.

Input	Naive Reads	Greedy Reads	Reduction
1	58	35	23 (39.7%)
2	98	56	42 (42.9%)
3	138	77	61 (44.2%)
4	90	51	39 (43.3%)
5	192	105	87 (45.3%)
6	658	350	308 (46.8%)
7	90	51	39 (43.3%)
8	2458	1295	1163 (47.3%)
9	4858	2555	2303 (47.4%)
Avg			43.9%

Figure 8. Naive vs Greedy on Prime

We picked `quicksort` and `prime` because they push on different parts of the allocator. Running both meant we could see whether the allocator handled different kinds of programs well, not just one.

For `quicksort`, it is a recursion-heavy program with a lot of function calls and array accesses. The function recurses on itself twice per call, and `main` calls `geti`, `puti`, and `putc` repeatedly while reading and writing into a 100-element array. Every call (recursive or not) makes the allocator flush its selected registers to the stack so the callee can use the temporaries however it wants, and every `array_load` and `array_store` hits memory. Because of this, `quicksort` is a good test for the parts of the allocator that deal with block boundaries: handling calls and branches, deciding what to load when a block starts, and flushing values back when it ends. If any of those were broken, `quicksort` would catch it pretty quickly since the program crosses block boundaries constantly.

For `prime`, it is the opposite. After some setup, it spends most of its time in one tight loop body that repeatedly checks divisibility, and the helper function `divisible` itself only has six instructions but gets called twice per iteration. Inside the main loop, the same handful of six virtual registers (being: `i`, `n`, `r`, `t`, `r0`, `rr`) get read over and over, and the loop body has long stretches of straight-line code between branches. This is exactly the case the greedy algorithm is built for. When the same virtual register gets read multiple times in one block, the naive allocator emits a separate `lw` for each read. Where the greedy allocator shines here is that the greedy allocator emits one load at the top of the block and reuses the physical register for the rest. So most of the savings on `prime` arise from benefits of providing the six variables six physical registers to use without reloading on every use, rather from anything happening at block boundaries like with `quicksort`.

Between the two, we get coverage of both sources of savings: keeping frequently used virtual registers in physical registers within a block, and handling the boundaries cleanly between blocks. The 42–44% reduction we got across both programs tells us the allocator is doing well on both fronts.

Performance Evaluation with Noteworthy Test Cases

Test	Naive Reads	Greedy Reads	Reduction
interleave_array	241	134	107 (-44.4%)
edge_arrays	263	159	104 (-39.5%)
bubble_sort	1165	621	544 (-46.7%)

Figure 9. Naive vs Greedy on Noteworthy Test Cases. Greedy outperforms Naive by a large margin due to the nature of these tests.

We found incredible results with `bubble_sort`, `interleave_array`, and `edge_arrays`. They had an incredible performance margin due to the nature of these tests. `bubble_sort` (–46.7%) had the biggest reduction. It sorts an 8-element array using nested loops, and the inner loop does two `array_load` operations, a comparison, and possibly two `array_store` operations every iteration. The same six virtual registers (`i`, `j`, `jj`, `ti`, `tj`, the array base) get read over and over inside one long block, which is exactly where the selected physical register pool pays off the most. One entry load ends up replacing dozens of per-instruction loads as Naive does. `interleave_array` (–44.4%) and `edge_arrays` (–39.5%) hit the same general pattern of a tight loop walking an array with the loop-control virtual registers reused every iteration. They land a bit lower than `bubble_sort` because their loop bodies and number of virtual registers are shorter (meaning greedy will show an improvement over naive, but it will be a little bit smaller but still be well above 30% reduction), so the fixed cost of an entry load is divided over less work.